MarutiX — Complete Developer & Operations Documentation

Version: 0.1.0 · **Last updated:** November 2026 · **Stack:** Next.js 16 · MongoDB · TypeScript · Docker

---

Table of Contents

1. Project Overview
   - 1.1 What is MarutiX?
   - 1.2 Key Features
   - 1.3 Tech Stack
   - 1.4 System Architecture
   - 1.5 Folder Structure
2. Prerequisites
3. Environment Setup
   - 3.1 Complete Environment Variable Reference
4. Local Development Setup
   - 4.1 Clone & Install
   - 4.2 MongoDB Setup
   - 4.3 Generate Secrets
   - 4.4 Minimum `.env.local`
   - 4.5 Run the App
   - 4.6 Grant Admin Access
5. Docker
   - 5.1 Images
   - 5.2 Docker Compose (Production)
   - 5.3 Commands Reference
   - 5.4 Troubleshooting
6. Database
   - 6.1 Collections Reference
   - 6.2 Indexes
   - 6.3 Encryption
   - 6.4 Backup & Restore
7. Integrations
   - 7.1 WhatsApp (Two Modes)
   - 7.2 OTP / Sign-In
   - 7.3 Email (SMTP)
   - 7.4 SMS (Twilio)
   - 7.5 AI (Anthropic)
   - 7.6 Maps & Discovery (OpenStreetMap)
   - 7.7 Webhooks
8. Development Guide
   - 8.1 Architecture & Data Flow
   - 8.2 API Reference
   - 8.3 Authentication

- 8.4 Background Workers
- 8.5 Security Headers

9. Testing
10. CI/CD & Deployment
11. Maintenance & Operations
    - 11.1 Logging
    - 11.2 Health Checks
    - 11.3 Backups
    - 11.4 Admin Operations
    - 11.5 Common Errors

---

1. Project Overview

1.1 What is MarutiX?

MarutiX is a **real-time emergency blood donation coordination platform** built for India. It connects people who urgently need blood with verified registered donors, and coordinates with nearby hospitals and blood banks through WhatsApp and email.

The core problem it solves: When a patient needs emergency blood, families post desperate messages on social media with no guarantees. MarutiX automates the entire coordination pipeline — from request intake, to donor matching, to alert delivery, to hospital outreach — in minutes rather than hours.

Who uses it: | Role | What they do | |—|—| | **Requesters** | Submit emergency blood requests via web form or WhatsApp bot | | **Donors** | Register, receive WhatsApp alerts, respond with availability | | **Admins** | Verify requests, trigger matching, manage outreach, monitor operations | | **Institutions** | Blood banks / hospitals receive outreach emails and report availability |

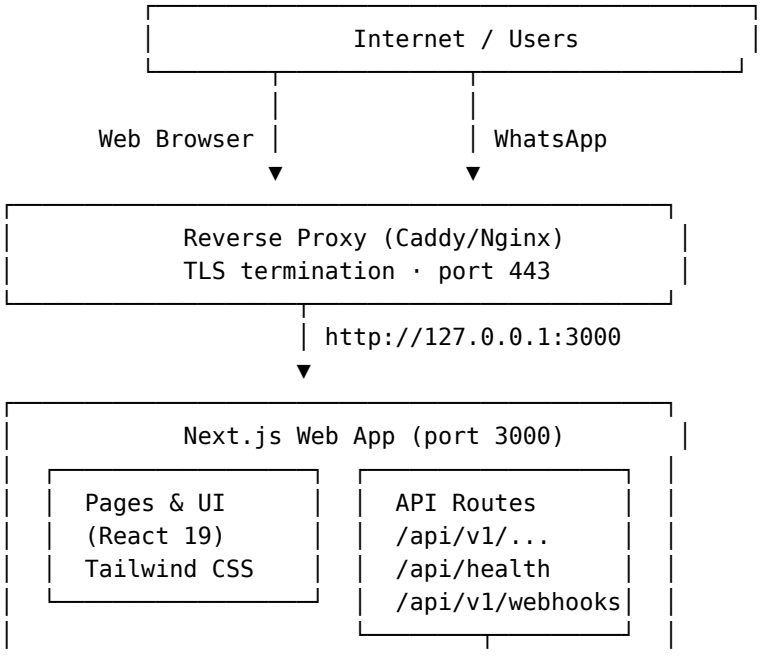
1.2 Key Features

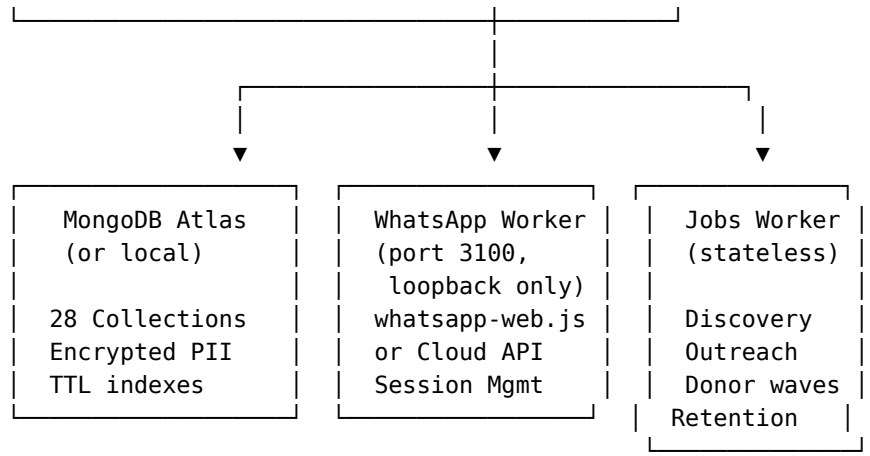
- **WhatsApp bot** — Donors register and receive alerts via conversational WhatsApp flows (official Cloud API or unofficial Web pilot)
- **Emergency request intake** — Web form and WhatsApp bot, PII encrypted at rest
- **AI-assisted bot** — Anthropic Claude understands natural-language responses (optional)
- **Automated donor matching** — Geo-aware, zone-wise wave alerts (5 km ⟶ 10 km ⟶ 25 km ⟶ 50 km)
- **Institution discovery** — Finds nearby blood banks via OpenStreetMap; sends outreach emails
- **Admin dashboard** — Full request lifecycle management, audit log, real-time WhatsApp status
- **MFA for admins** — TOTP (Google Authenticator), encrypted secrets
- **Privacy-first** — Phone numbers stored only as keyed hashes + AES-256-GCM ciphertext; request PII auto-anonymized after `REQUEST_RETENTION_DAYS`
- **Share links** — Tokenised tracking URLs for requesters (never expose patient data)
- **Consent management** — DPDP Act–aligned consent records, opt-out preserved forever
- **Production Docker** — Multi-stage Dockerfile, Compose with security hardening

1.3 Tech Stack

Layer	Technology	Version
Frontend	Next.js (App Router, Turbopack)	16.4.5
UI	React	19.3.0
Styling	Tailwind CSS	4
Backend	Next.js API Routes (server-only)	—
Language	TypeScript	5
Validation	Zod	4.6.7
Database	MongoDB via Mongoose	9.11.4
WhatsApp (official)	Meta Cloud API	Graph v26.0
WhatsApp (pilot)	whatsapp-web.js	1.34.7
Email	Nodemailer (SMTP)	10.3.1
SMS	Twilio	—
AI	Anthropic SDK (Claude)	1.4.0
QR Codes	qrcode	1.5.4
Testing	Vitest	5.1.0
Runtime	Node.js	24
Containers	Docker + Compose	—
CI/CD	GitHub Actions	—
Registry	GitHub Container Registry (GHCR)	—

1.4 System Architecture





Key architectural decisions: - The WhatsApp worker runs as a **separate long-lived process** with a MongoDB-backed lease (only one can hold the WhatsApp session at a time) - The jobs worker is **stateless and horizontally scalable** — jobs are claimed atomically from MongoDB - All PII (phone numbers, patient names, contact details) is **encrypted at rest** with AES-256-GCM using `PII_ENCRYPTION_KEY` - Phone numbers are additionally stored as **HMAC-SHA256 keyed hashes** (`phoneLookup`) for indexed lookups without decrypting - The admin dashboard talks to the WhatsApp worker via a **loopback-only control HTTP server** (port 3100); never directly exposed

1.5 Folder Structure

```
MauritX/
├── src/
│   ├── app/                    # Next.js App Router
│   │   ├── (auth)/login/       # Login page
│   │   ├── (donor)/            # Donor dashboard, profile
│   │   ├── (public)/           # Public pages: donate, get-help, track,
respond, privacy
│   │   ├── admin/              # Admin dashboard pages
│   │   └── api/                # API route handlers
│   │       ├── auth/           # Legacy auth (OTP send/verify/logout)
│   │       ├── health/         # GET /api/health
│   │       ├── v1/             # Versioned REST API
│   │       │   ├── auth/       # OTP, MFA
│   │       │   ├── donors/     # Donor profile, availability, consents
│   │       │   ├── requests/   # Emergency requests, share links
│   │       │   ├── admin/      # Admin-only APIs
│   │       │   └── webhooks/   # WhatsApp Cloud API webhook
│   │       └── whatsapp/       # Legacy WhatsApp control (deprecated by /v1/
admin/whatsapp)
│   ├── features/               # Domain types and API clients (shared between
client & server)
│   │   ├── auth/
│   │   ├── coordination/
│   │   ├── donors/
│   │   ├── requests/
│   │   └── whatsapp/
│   ├── lib/                    # Pure library code (server-side unless noted)
```

```
│   │   ├── ai/config.ts        # AI (Anthropic) config
│   │   ├── auth/               # JWT, permissions, session helpers
│   │   ├── coordination/config.ts  # Alert & matching parameters
│   │   ├── db/mongoose.ts      # MongoDB connection
│   │   ├── discovery/config.ts # OpenStreetMap config
│   │   ├── email/config.ts     # SMTP config
│   │   ├── env.ts              # Core env schema (Zod)
│   │   ├── logger.ts           # Structured JSON logger (pino-compatible)
│   │   ├── privacy/            # Retention config
│   │   ├── security/           # crypto, config-check, TOTP, redact
│   │   ├── sms/config.ts       # Twilio config
│   │   ├── validation/phone.ts # Indian mobile number normalizer
│   │   ├── whatsapp/           # Cloud API client & config
│   │   └── whatsapp-web/       # Web pilot config & control client
│   ├── server/                 # Server-only business logic
│   │   ├── admin/              # Admin services (coordination, operations)
│   │   ├── ai/                 # Anthropic client & interpretation
│   │   ├── conversations/      # Bot conversation state machines
│   │   ├── institutions/       # Institution discovery & service
│   │   ├── jobs/               # Job queue runner
│   │   ├── messaging/          # Outbox dispatcher (Cloud + Web)
│   │   ├── models/             # Mongoose schemas (28 collections)
│   │   ├── monitoring/         # Error reporting
│   │   ├── requests/           # Request creation (HTTP + WhatsApp)
│   │   ├── services/           # Retention anonymization
│   │   ├── settings/           # System settings service
│   │   └── whatsapp/           # Connection store, lease management
│   └── worker/                 # Long-lived worker entry points
│       ├── jobs/main.ts        # Jobs worker
│       └── whatsapp/           # WhatsApp Web pilot worker
│           ├── main.ts
│           ├── control-server.ts
│           ├── session-dir.ts
│           └── session-manager.ts
├── tests/                      # Vitest integration & unit tests
│   ├── setup.ts                # Test environment bootstrap
│   ├── helpers/fixtures.ts
│   └── *.integration.test.ts
├── scripts/                    # Operator scripts
│   ├── admin-role.ts           # Grant/revoke admin, MFA reset
│   ├── audit-gate.mjs          # CI dependency security gate
│   ├── backup.sh               # MongoDB logical backup
│   ├── restore-test.sh         # Backup restore verification
│   ├── verify-restore.ts       # Collection count verifier
│   └── ci/                     # CI scripts
│       ├── smoke-web-image.sh
│       ├── smoke-worker-image.sh
│       └── remote-deploy.sh
├── deploy/
│   └── docker-compose.prod.yml # Production Docker Compose
├── .github/
│   ├── audit-allowlist.json    # Dep vulnerability allowlist
│   └── workflows/
```

```
│       ├── code-quality.yml    # Stage 1: ESLint, TS, tests, audit, CodeQL
│       ├── build-push.yml      # Stage 2: Build, Docker images, push to GHCR
│       └── deploy.yml          # Stage 3: SSH deploy, health check, rollback
├── Dockerfile                  # Web app image (multi-stage)
├── Dockerfile.worker           # WhatsApp/jobs worker image
├── next.config.ts              # Next.js config (CSP, security headers)
├── package.json
└── tsconfig.json
```

---

2. Prerequisites

Before you start, install the following on your machine:

Required software

Software	Minimum version	Install command (macOS)	Purpose
Node.js	24 LTS	`brew install node`	Runtime for all processes
npm	10+	Bundled with Node.js	Package manager
Git	2.x	`brew install git`	Source control
MongoDB	7.0+	See §4.2	Database (or use Atlas)
mongosh	2.x	`brew install mongosh`	MongoDB shell
MongoDB Tools	100.x	`brew install mongodb-database-tools`	`mongodump`/`mongorestore` for backups

Optional but recommended

Software	Purpose	Install
Docker Desktop	Run production stack locally	https://www.docker.com/products/docker-desktop
actionlint	Validate GitHub Actions workflows	`brew install actionlint`
shellcheck	Shell script linting	`brew install shellcheck`

Required accounts (production only)

Service	Purpose	URL
MongoDB Atlas	Cloud database	https://cloud.mongodb.com
GitHub	Source + GHCR image registry	https://github.com
Meta Developer Account	WhatsApp Cloud API	https://developers.facebook.com
Anthropic	AI (optional)	https://console.anthropic.com
Twilio	SMS OTP fallback (optional)	https://twilio.com
SMTP provider	Institution outreach email	Any (Gmail, SendGrid, Resend, etc.)
VPS / server	Hosting	Any (DigitalOcean, Hetzner, etc.)

Version check

```
node --version    # Must be ≥ 24.0.0
npm --version     # Must be ≥ 10.0.0
mongosh --version # Must be ≥ 2.0.0
docker --version  # Optional, any recent version
```

---

3. Environment Setup

MarutiX is configured entirely through environment variables. In development, they go in `.env.local` (never commit this file). In production, they go in `/opt/marutix/.env.production` on the server (chmod 600).

> **Important:** Every startup call to `validateServerConfiguration()` will throw a detailed error listing every missing or malformed variable before the app accepts any traffic.

3.1 Complete Environment Variable Reference

Core (always required)

Variable	Required	Default	Description
`MONGODB_URI`	✅	—	Full MongoDB connection string. Local: `mongodb://127.0.0.1:27017/marutix`. Atlas: `mongodb+srv://user:pass@cluster.mongodb.net/marutix?retryWrites=true`
`APP_BASE_URL`	✅	—	Full public URL of the app. Dev: `http://localhost:3000`. Prod: `https://marutix.example.com`
`AUTH_JWT_SECRET`	✅	—	Secret for signing session JWTs. Must be ≥ 32 characters, cryptographically random. Generate: `openssl rand -hex 64`
`OTP_HASH_SECRET`	✅	—	HMAC key for hashing OTP codes before storage. Must be ≥ 32 characters. Generate: `openssl rand -hex 32`
`PII_ENCRYPTION_KEY`	✅	—	32-byte AES-256 key encoded as base64. Used to encrypt all phone numbers and patient details. Generate: `openssl rand -base64 32`
`NODE_ENV`	—	`development`	`development`, `test`, or `production`. Set to `production` in Docker.

OTP delivery

Variable	Required	Default	Description
`OTP_PROVIDER`	—	Auto-detected	`whatsapp` (send OTP via WhatsApp) or `development` (logs code to server console; local dev only). Auto-selects `whatsapp` if WhatsApp credentials are present.
`WHATSAPP_ACCESS_TOKEN`	Required for WhatsApp OTP	—	Meta Graph API token for sending OTP template messages
`WHATSAPP_PHONE_NUMBER_ID`	Required for WhatsApp OTP	—	Numeric Phone Number ID from Meta Business Manager
`WHATSAPP_OTP_TEMPLATE_NAME`	Required for WhatsApp OTP	—	Name of the approved OTP message template
`WHATSAPP_OTP_TEMPLATE_LANGUAGE`	—	`en`	Language code for the OTP template
`WHATSAPP_GRAPH_API_VERSION`	—	`v26.0`	Meta Graph API version (format: `v26.0`)

Messaging provider (WhatsApp outbox)

Variable	Required	Default	Description
`MESSAGING_PROVIDER`	—	`whatsapp_web`	`whatsapp_web` (unofficial pilot) or `whatsapp_cloud` (official Cloud API)
`WHATSAPP_APP_SECRET`	Required for `whatsapp_cloud`	—	App secret from Meta App Dashboard → App settings → Basic. Used to verify webhook signatures.
`WHATSAPP_WEBHOOK_VERIFY_TOKEN`	Required for `whatsapp_cloud`	—	A random secret you choose; set the same value in the Meta webhook configuration
`WHATSAPP_TEMPLATE_EMERGENCY_ALERT`	—	—	Name of the approved emergency alert message template
`WHATSAPP_TEMPLATE_REQUEST_CONFIRMATION`	—	—	Name of the approved request confirmation template
`WHATSAPP_TEMPLATE_LANGUAGE`	—	`en`	Template language code
`WHATSAPP_CLOUD_MAX_MESSAGES_PER_HOUR`	—	`250`	Rate limit for business-initiated messages
`WHATSAPP_CLOUD_MAX_REPLIES_PER_HOUR`	—	`1000`	Rate limit for replies within 24-hour window
`WHATSAPP_CLOUD_MIN_SEND_INTERVAL_MS`	—	`250`	Minimum milliseconds between messages

WhatsApp Web pilot worker

Variable	Required	Default	Description
`WHATSAPP_WEB_WORKER_SECRET`	✅	—	Bearer token for the loopback control server (port 3100). Generate: `openssl rand -hex 32`
`WHATSAPP_WEB_WORKER_URL`	—	`http://127.0.0.1:3100`	URL the web app uses to reach the WhatsApp worker
`WHATSAPP_WEB_SESSION_DIR`	—	`.data/whatsapp-session`	Directory for Puppeteer session persistence
`WHATSAPP_WEB_DEVICE_NAME`	—	`MarutiX`	Device name shown in WhatsApp Linked Devices
`WHATSAPP_WEB_PUBLIC_NUMBER`	—	—	WhatsApp number shown in “Become a Donor” public links (baked into Docker image at build time)
`WHATSAPP_WEB_TEST_RECIPIENTS`	—	—	Comma-separated list of phone numbers that can receive messages in dev/staging (restricts blast to these numbers)
`WHATSAPP_WEB_MAX_MESSAGES_PER_HOUR`	—	`100`	Message rate limit
`WHATSAPP_WEB_MAX_REPLIES_PER_HOUR`	—	`500`	Reply rate limit
`WHATSAPP_WEB_MIN_SEND_INTERVAL_MS`	—	`500`	Minimum milliseconds between messages
`WHATSAPP_WEB_MESSAGE_TTL_MINUTES`	—	`30`	Drop outbox messages older than this
`WHATSAPP_WEB_NO_SANDBOX`	—	—	Set `true` in Docker (Chromium sandbox needs privileges the container doesn’t have)
`PUPPETEER_EXECUTABLE_PATH`	—	—	Path to Chromium binary. Docker sets this to `/usr/bin/chromium`
`PUPPETEER_SKIP_DOWNLOAD`	—	—	Set `true` to skip Chrome download (web image; Docker always sets this)

Email (SMTP)

Variable	Required	Default	Description
`EMAIL_PROVIDER`	—	`none`	`none` (email disabled) or `smtp`
`SMTP_HOST`	Required if `EMAIL_PROVIDER=smtp`	—	SMTP server hostname (e.g., `smtp.gmail.com`)
`SMTP_PORT`	—	`587`	SMTP port (587 for STARTTLS, 465 for SSL)
`SMTP_SECURE`	—	`false`	`true` for port 465 (implicit TLS)
`SMTP_USER`	—	—	SMTP authentication username
`SMTP_PASSWORD`	—	—	SMTP authentication password
`EMAIL_FROM`	Required if `EMAIL_PROVIDER=smtp`	—	Sender address: `"MarutiX" <noreply@example.com>`
`EMAIL_REPLY_TO`	—	—	Reply-to address
`EMAIL_MAX_PER_HOUR`	—	`60`	Rate limit for outbound emails

SMS (Twilio — optional OTP fallback)

Variable	Required	Default	Description
`SMS_PROVIDER`	—	`none`	`none` or `twilio`
`TWILIO_ACCOUNT_SID`	Required if `SMS_PROVIDER=twilio`	—	Twilio Account SID (starts with `AC`)
`TWILIO_AUTH_TOKEN`	Required if `SMS_PROVIDER=twilio`	—	Twilio Auth Token
`TWILIO_MESSAGING_SERVICE_SID`	—	—	Twilio Messaging Service SID (recommended over `TWILIO_FROM`)
`TWILIO_FROM`	—	—	Twilio phone number (if no Messaging Service SID)
`SMS_OTP_BODY_TEMPLATE`	Required if `SMS_PROVIDER=twilio`	—	DLT-registered template text with `{code}` placeholder
`OTP_SMS_DAILY_CAP`	—	`500`	Max SMS OTPs sent per UTC day across all users
`OTP_SMS_PER_PHONE_DAILY`	—	`3`	Max SMS OTPs per phone number per day

AI (Anthropic — optional)

Variable	Required	Default	Description
`AI_PROVIDER`	—	`none`	`none` or `anthropic`
`ANTHROPIC_API_KEY`	Required if `AI_PROVIDER=anthropic`	—	Anthropic API key (from console.anthropic.com)
`AI_MODEL`	—	`claude-opus-5-5`	Anthropic model name
`AI_TIMEOUT_MS`	—	`10000`	AI call timeout in milliseconds
`AI_MAX_CALLS_PER_NUMBER_PER_HOUR`	—	`20`	Rate limit per WhatsApp number

Coordination & matching

Variable	Required	Default	Description
ALERT_BATCH_SIZE	—	5	Donors alerted per wave step
ALERT_MAX_PER_REQUEST	—	40	Total donors alerted per request
ALERT_MAX_PER_DONOR_PER_DAY	—	2	Alerts a single donor can receive per day
ALERT_TTL_MINUTES	—	60	Alert expiry
MATCH_MIN_DONATION_INTERVAL_DAYS	—	90	Minimum days between donations (eligibility filter)
MATCH_MAX_CANDIDATES	—	100	Maximum donor candidates per matching run
COORDINATION_AUTOSTART	—	true	Auto-start coordination when admin activates a request
DONOR_ZONES_KM	—	5,10,25,50	Zone radii for progressive donor outreach
DONOR_WAVE_SIZE	—	5	Donors per wave
DONOR_WAVE_INTERVAL_MINUTES	—	20	Minutes between waves
DONOR_TARGET_INTERESTED	—	0 (= 2× units)	Stop expanding after this many interested donors
OUTREACH_RADIUS_KM	—	25	Radius for institution discovery
OUTREACH_MAX_INSTITUTIONS	—	10	Max institutions to contact per request
OUTREACH_INCLUDE_UNREVIEWED	—	false	Include unreviewed institutions in outreach
OUTREACH_MAX_PER_INSTITUTION_PER_DAY	—	5	Anti-spam limit
COORDINATOR_CONTACT_PHONE	—	—	Coordinator's phone shown in alerts
COORDINATOR_CONTACT_EMAIL	—	—	Coordinator's email shown in outreach

Institution discovery (OpenStreetMap)

Variable	Required	Default	Description
`DISCOVERY_PROVIDER`	—	`none`	`none` or `openstreetmap`
`DISCOVERY_CONTACT_EMAIL`	Required if `openstreetmap`	—	Your email (required by OpenStreetMap usage policy)
`OVERPASS_URL`	—	`https://overpass-api.de/api/interpreter`	Overpass API endpoint
`NOMINATIM_URL`	—	`https://nominatim.openstreetmap.org`	Nominatim geocoding endpoint
`DISCOVERY_WEBSITE_ENRICHMENT`	—	`true`	Fetch institution websites for enrichment
`DISCOVERY_TIMEOUT_MS`	—	`60000`	Discovery request timeout

Privacy & compliance

Variable	Required	Default	Description
`REQUEST_RETENTION_DAYS`	—	`180`	Days before closed requests are anonymized (min 7, max 3650)
`TRUSTED_PROXY_HOPS`	—	`1`	Number of reverse proxies in front of the app (for correct IP extraction)

Security & monitoring

Variable	Required	Default	Description
`ERROR_ALERT_WEBHOOK_URL`	—	—	HTTPS webhook URL for error alerts (Slack, Discord, etc.). Must be https.
`DEV_ALLOWED_ORIGINS`	—	—	Comma-separated hostnames for LAN dev access (dev only)

4. Local Development Setup

4.1 Clone & Install

```
# Clone the repository
git clone https://github.com/Piyush-Sharma788/MauritX.git
cd MauritX

# Install all dependencies (this takes 2–5 min on first run)
npm install
```

> **Note:** The `postinstall` script downloads Chromium for Puppeteer unless `PUPPETEER_SKIP_DOWNLOAD=true` is set. For the web-only setup, you can skip the download:
>
> ```
> PUPPETEER_SKIP_DOWNLOAD=true npm install
> ```

4.2 MongoDB Setup

Option A — Local MongoDB (easiest for development)

```
# macOS (Homebrew)
brew tap mongodb/brew
brew install mongodb-community@8.0
brew services start mongodb-community@8.0

# Verify it's running
mongosh --eval "db.adminCommand({ ping: 1 })"
# Expected: { ok: 1 }
```

Your `MONGODB_URI` will be: `mongodb://127.0.0.1:27017/marutix`

Option B — MongoDB Atlas (recommended for production)

1. Go to https://cloud.mongodb.com and create a free account
2. Create a free **M0** cluster
3. Under **Database Access**, create a database user with read/write access
4. Under **Network Access**, add your IP address (or `0.0.0.0/0` for all)
5. Click **Connect → Connect your application** and copy the connection string
6. Replace `<password>` with your user's password

Your `MONGODB_URI` will look like: `mongodb+srv://myuser:mypassword@cluster0.abc123.mongodb.net/marutix?retryWrites=true&w=majority`

4.3 Generate Secrets

Run these commands to generate cryptographically secure secrets:

```
# AUTH_JWT_SECRET (64-byte hex = 128 chars)
openssl rand -hex 64

# OTP_HASH_SECRET (32-byte hex = 64 chars)
openssl rand -hex 32

# PII_ENCRYPTION_KEY (32-byte base64 = ~44 chars)
openssl rand -base64 32
```

```
# WHATSAPP_WEB_WORKER_SECRET (32-byte hex = 64 chars)
openssl rand -hex 32
```

> **Warning:** Never reuse the same value for different secrets. If any two secrets are identical, startup validation will reject the configuration.

4.4 Minimum `.env.local`

Create `.env.local` in the project root (never commit this file — it's already in `.gitignore`):

```
# .env.local — Development configuration
# Replace all placeholder values with your actual generated secrets

# --- Core ---
MONGODB_URI=mongodb://127.0.0.1:27017/marutix
APP_BASE_URL=http://localhost:3000
AUTH_JWT_SECRET=<output of: openssl rand -hex 64>
OTP_HASH_SECRET=<output of: openssl rand -hex 32>
PII_ENCRYPTION_KEY=<output of: openssl rand -base64 32>

# --- OTP: development mode (logs OTP to server console, no WhatsApp needed)
---
OTP_PROVIDER=development

# --- WhatsApp Web worker (required even without WhatsApp, for config
validation) ---
WHATSAPP_WEB_WORKER_SECRET=<output of: openssl rand -hex 32>
```

This minimal configuration: - ✅ Lets you run the web app and admin dashboard - ✅ Sign-in OTPs appear in the terminal (no WhatsApp needed) - ❌ No WhatsApp bot (need to run the worker separately) - ❌ No email outreach - ❌ No SMS OTP - ❌ No AI-enhanced bot

4.5 Run the App

Terminal 1 — Web app (Next.js dev server)

```
npm run dev
# Open: http://localhost:3000
```

Terminal 2 — WhatsApp Web worker (optional)

Only needed if you have a WhatsApp account to link:

```
npm run whatsapp:worker
# The worker will log a QR code URL; scan it with the WhatsApp app
# Check status at: http://localhost:3000/admin/whatsapp (after granting admin
access)
```

Terminal 3 — Jobs worker (optional)

Runs institution discovery, outreach emails, and donor wave scheduling:

```
npm run jobs:worker
```

4.6 Grant Admin Access

After signing in with your phone number:

```
# Grant admin role to a phone number
npm run admin:grant -- +919876543210

# List all admins
npm run admin:list

# Revoke admin role
npm run admin:revoke -- +919876543210

# Reset admin MFA (if locked out)
npm run admin:mfa-reset -- +919876543210
```

Admin dashboard: http://localhost:3000/admin

---

5. Docker

5.1 Images

MarutiX uses two Docker images, both based on `node:24-bookworm-slim`:

`marutix-web` (Dockerfile)

A **3-stage** production image: 1. **`deps` stage** — Installs npm dependencies with build cache mount 2. **`build` stage** — Runs `next build` with `NEXT_OUTPUT=standalone` 3. **`runner` stage** — Copies only the standalone output (~100 MB)

Runs as the unprivileged `node` user. Listens on port 3000.

```
# Build locally
docker build \
  --build-arg WHATSAPP_WEB_PUBLIC_NUMBER="+919876543210" \
  -t marutix-web:latest \
  -f Dockerfile .
```

`marutix-worker` (Dockerfile.worker)

A single-stage image that installs: - System Chromium (from Debian repos) — avoids Puppeteer's download - Noto fonts (Indic scripts + emoji) - Full npm install (tsx for running TypeScript) - All `src/` and `scripts/` files

Runs as the unprivileged `node` user. Session data on `/app/.data` volume.

```
# Build locally
docker build -t marutix-worker:latest -f Dockerfile.worker .
```

5.2 Docker Compose (Production)

The production stack is defined in `deploy/docker-compose.prod.yml`.

It runs **three services**:

Service	Image	Port	Purpose
`web`	`marutix-web`	`127.0.0.1:3000`	Next.js app (behind reverse proxy)
`worker`	`marutix-worker`	Shared with `web`	WhatsApp Web pilot (loopback network)
`jobs`	`marutix-worker` (different cmd)	—	Background jobs (discovery, email, donor waves)

Shared security settings (applied to all services): - `restart: unless-stopped` - `no-new-privileges: true` - `cap_drop: ALL` - `env_file: .env.production` - JSON file logging with rotation (10 MB × 5 files)

The worker shares the web container's **network namespace** (`network_mode: "service:web"`) so its control HTTP server on port 3100 is only accessible via loopback, never on the container network.

5.3 Commands Reference

```
# --- Build both images ---
docker build --build-arg WHATSAPP_WEB_PUBLIC_NUMBER="+91..." -t marutix-
web:v1.0.0 -f Dockerfile .
docker build -t marutix-worker:v1.0.0 -f Dockerfile.worker .

# --- Run production stack ---
# (Assumes .env.production exists in current directory)
MARUTIX_IMAGE_PREFIX=ghcr.io/your-username \
MARUTIX_IMAGE_TAG=v1.0.0 \
docker compose -f deploy/docker-compose.prod.yml up -d

# --- Check status ---
docker compose -f deploy/docker-compose.prod.yml ps
docker compose -f deploy/docker-compose.prod.yml logs -f web
docker compose -f deploy/docker-compose.prod.yml logs -f worker
docker compose -f deploy/docker-compose.prod.yml logs -f jobs

# --- Stop stack ---
docker compose -f deploy/docker-compose.prod.yml down

# --- Rolling update (new tag) ---
MARUTIX_IMAGE_PREFIX=ghcr.io/your-username \
MARUTIX_IMAGE_TAG=v1.1.0 \
docker compose -f deploy/docker-compose.prod.yml up -d --no-deps web

# --- Open a shell inside the web container ---
docker compose -f deploy/docker-compose.prod.yml exec web sh

# --- Inspect health ---
docker inspect --format='{{json .State.Health}}' marutix-web-1 | python3 -m
json.tool
```

5.4 Troubleshooting

Symptom	Likely cause	Fix
Container exits immediately	Config validation failed	Check `docker logs marutix-web-1` for "refused to start" message; fix the named variable
`/api/health` returns `{"db":"down"}`	MongoDB not reachable	Check `MONGODB_URI`; verify Atlas IP whitelist
Worker exits with "lease held"	Another worker is running	Stop the duplicate; or delete `whatsapp_connection` document and restart
QR code never appears	Session directory not writable	Check volume mount permissions; run `chown -R node:node /app/.data`
OTP never arrives	`OTP_PROVIDER` misconfigured	In dev, set `OTP_PROVIDER=development` and read the OTP from server logs
`Error: WHATSAPP_GRAPH_API_VERSION must look like v26.0`	Bad env value	Format must be `v26.0`, not `26.0` or `v26`

---

6. Database

6.1 Collections Reference

MarutiX uses **28 MongoDB collections**:

Collection	Model	Description
users	User	All users (donors, admins, institution users). Phone stored as encrypted + keyed hash.
donor_profiles	DonorProfile	Blood group, city, geolocation (2dsphere), availability flag, notification consent
emergency_requests	EmergencyRequest	Blood requests. Patient name, contact phone encrypted. Location as GeoJSON point.
request_responses	RequestResponse	Donor responses to alerts (interested, unavailable, etc.)
request_share_links	RequestShareLink	Tokenized tracking links for requesters
sessions	Session	Active JWT sessions with device info and expiry
otp_challenges	OtpChallenge	In-flight OTP verification records (TTL indexed)
rate_limit_buckets	RateLimitBucket	Per-phone sliding window counters (TTL indexed)
idempotency_records	IdempotencyRecord	Request deduplication for critical mutations
audit_logs	AuditLog	Immutable audit trail of all security and operational events
consent_records	ConsentRecord	Append-only DPDP consent history
messaging_opt_outs	MessagingOptOut	Permanent opt-outs (never deleted)
notification_events	NotificationEvent	Outbox: messages queued for WhatsApp delivery
contact_access_logs	ContactAccessLog	Records every time a donor's contact details are revealed
matching_runs	MatchingRun	Records of each donor matching execution
coordination_events	CoordinationEvent	Timeline of automation steps for each request
institutions	Institution	Hospitals, blood banks, NGOs, and other partner organizations

6.2 Indexes

Key indexes for performance:

```
users:              phoneLookup (unique), role, status
donor_profiles:     userId (unique), bloodGroup+availableForRequests, location
(2dsphere)
emergency_requests: publicRequestId (unique), status, bloodGroup+city,
location (2dsphere)
sessions:           token (unique), userId, expiresAt (TTL)
otp_challenges:     phone+type, expiresAt (TTL)
rate_limit_buckets: key+windowStart (TTL)
whatsapp_conversations: phoneLookup (unique), userId, expiresAt (TTL)
whatsapp_inbound_messages: providerMessageId (unique), expiresAt (TTL),
phoneLookup
jobs:               status+scheduledAt (compound), claimedBy+type (sparse)
audit_logs:         userId, action, createdAt
consent_records:    userId+purpose, createdAt
```

TTL indexes automatically clean up: - Expired OTP challenges - Expired rate limit buckets - Old WhatsApp conversations - Processed inbound message IDs - Old bot metric days

6.3 Encryption

All PII uses **two-layer protection**:

1. **Keyed hash** (`phoneLookup`): `HMAC-SHA256(phone, OTP_HASH_SECRET)` — enables indexed lookups without decryption
2. **Encryption** (`phoneEncrypted`, `patientNameEncrypted`, etc.): `AES-256-GCM` using `PII_ENCRYPTION_KEY`

> **Critical:** If you lose `PII_ENCRYPTION_KEY`, all encrypted data is permanently unreadable. Back it up separately from the database.

Admin TOTP secrets are also encrypted. Recovery code hashes are keyed hashes (not reversible).

6.4 Backup & Restore

Create a backup

```bash
# Basic backup (saves to /var/backups/marutix by default)
MONGODB_URI=mongodb://127.0.0.1:27017/marutix \
BACKUP_DIR=/var/backups/marutix \
bash scripts/backup.sh

# Encrypted backup (recommended for production)
echo "my-strong-passphrase" > /etc/marutix-backup.key
chmod 600 /etc/marutix-backup.key
MONGODB_URI=mongodb+srv://... \
BACKUP_ENCRYPTION_PASSPHRASE_FILE=/etc/marutix-backup.key \
bash scripts/backup.sh

# Point-in-time backup (requires MongoDB replica set or Atlas)
BACKUP_OPLOG=1 MONGODB_URI=... bash scripts/backup.sh
```

Output files per run: - `marutix-20261105T123456Z.archive.gz` — gzip-compressed mongodump archive - `marutix-20261105T123456Z.archive.gz.sha256` — checksum for integrity verification - `marutix-20261105T123456Z.manifest.json` — collection counts at backup time

Retention: Old backups older than `BACKUP_RETENTION_DAYS` (default 14) are pruned automatically.

Verify a backup

```
# Verify checksum and restore to a temporary MongoDB instance
MONGODB_URI=mongodb://127.0.0.1:27017/marutix_restore_test \
bash scripts/restore-test.sh /var/backups/marutix/
marutix-20261105T123456Z.archive.gz
```

Restore from backup

```
# Restore to a database (WARNING: this overwrites all data)
mongorestore \
  --uri "mongodb://127.0.0.1:27017" \
  --db marutix_restored \
  --archive=/var/backups/marutix/marutix-20261105T123456Z.archive.gz \
  --gzip

# Restore from encrypted backup
openssl enc -d -aes-256-cbc -pbkdf2 -iter 600000 \
  -pass file:/etc/marutix-backup.key \
  -in marutix-....archive.gz.enc \
  | mongorestore --uri "..." --db marutix --archive --gzip
```

---

7. Integrations

7.1 WhatsApp (Two Modes)

MarutiX supports two WhatsApp integration modes controlled by `MESSAGING_PROVIDER`:

Mode A: WhatsApp Web Pilot (`MESSAGING_PROVIDER=whatsapp_web`)

Uses the unofficial whatsapp-web.js library. **For controlled pilots only** — this is not an officially supported integration.

How it works: 1. Admin scans a QR code at `/admin/whatsapp` 2. The WhatsApp Web worker manages the browser session via Puppeteer + Chromium 3. The worker holds a MongoDB lease (only one worker at a time) 4. Inbound messages trigger the bot conversation engine 5. Outbound messages are queued in `notification_events` and dispatched with pacing

Setup: 1. Set `WHATSAPP_WEB_WORKER_SECRET` (generate a random secret) 2. Run `npm run whatsapp:worker` 3. Scan the QR code at the admin dashboard 4. The session is saved to `WHATSAPP_WEB_SESSION_DIR` for auto-reconnect on restart

Mode B: WhatsApp Cloud API (`MESSAGING_PROVIDER=whatsapp_cloud`)

Official Meta Business Platform integration. Required for production at scale.

Setup steps: 1. Create a Meta Developer account at https://developers.facebook.com 2. Create a Business App with WhatsApp product 3. Get your **Phone Number ID** and

Access Token from the WhatsApp API setup 4. Create a **message template** for OTP delivery (must be approved by Meta) 5. Configure a **webhook** at `https://yourdomain.com/api/v1/webhooks/whatsapp` - Verify Token: set `WHATSAPP_WEBHOOK_VERIFY_TOKEN` to any random secret - Subscribe to: `messages` events 6. Create a template named in `WHATSAPP_TEMPLATE_EMERGENCY_ALERT` for donor alerts

Required env vars for Cloud API:

```
MESSAGING_PROVIDER=whatsapp_cloud
WHATSAPP_ACCESS_TOKEN=EAAxxxxxxxxxxx
WHATSAPP_PHONE_NUMBER_ID=1234567890123
WHATSAPP_APP_SECRET=abc123def456...
WHATSAPP_WEBHOOK_VERIFY_TOKEN=<your random secret>
WHATSAPP_TEMPLATE_EMERGENCY_ALERT=emergency_alert
WHATSAPP_TEMPLATE_REQUEST_CONFIRMATION=request_confirmation
```

Bot conversation flows

The WhatsApp bot handles these conversation flows:

Flow	Trigger	Description
`donor_onboarding`	"Register" / "Donate"	Collects blood group, city, locality, geolocation consent
`donor_update`	"Update" / "Change"	Updates donor profile
`emergency_request`	"Need blood"	Creates an emergency request via WhatsApp
`request_cancel`	"Cancel"	Cancels a request the user created
`donor_response`	Received an alert	Donor replies with availability

With `AI_PROVIDER=anthropic`, the bot uses Claude to understand natural-language messages that don't match fixed commands, and routes them to the correct flow.

7.2 OTP / Sign-In

MarutiX uses **phone-based OTP authentication** (no passwords):

1. User enters phone number → `POST /api/v1/auth/otp/request`
2. OTP generated, hashed with `OTP_HASH_SECRET`, stored in `otp_challenges`
3. OTP delivered via WhatsApp template or SMS (Twilio) or logged to console (dev)
4. User enters OTP → `POST /api/v1/auth/otp/verify`
5. Session JWT created, stored in `sessions`, set as HTTP-only cookie

Rate limiting: Max OTP attempts per phone, per hour and per day (sliding window, stored in `rate_limit_buckets`).

MFA (admins only): After OTP login, admin users with MFA enabled must verify a TOTP code from Google Authenticator. MFA enrollment at `/admin/mfa`.

7.3 Email (SMTP)

Email is used **only for institution outreach** (not for OTP or user notifications).

```
# Example: Gmail SMTP (use an App Password, not your account password)
EMAIL_PROVIDER=smtp
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=your@gmail.com
SMTP_PASSWORD=xxxx-xxxx-xxxx-xxxx   # Google App Password
EMAIL_FROM="MarutiX" <your@gmail.com>
EMAIL_MAX_PER_HOUR=30

# Example: Resend
SMTP_HOST=smtp.resend.com
SMTP_PORT=587
SMTP_USER=resend
SMTP_PASSWORD=re_xxxxxxxxxxx
EMAIL_FROM=noreply@yourdomain.com
```

7.4 SMS (Twilio)

Optional fallback for OTP delivery when WhatsApp is unavailable.

> **India-specific requirement:** All promotional and transactional SMS must comply with TRAI/DLT regulations. Register your sender ID and message template on the DLT platform through Twilio before enabling SMS.

```
SMS_PROVIDER=twilio
TWILIO_ACCOUNT_SID=ACxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
TWILIO_AUTH_TOKEN=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
TWILIO_MESSAGING_SERVICE_SID=MGxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
# Template must be DLT-registered:
SMS_OTP_BODY_TEMPLATE=Your MarutiX sign-in code is {code}. Valid for {minutes}
minutes.
```

7.5 AI (Anthropic)

When enabled, the WhatsApp bot passes ambiguous messages to Claude for interpretation:

```
AI_PROVIDER=anthropic
ANTHROPIC_API_KEY=sk-ant-api03-xxxxxxxx
AI_MODEL=claude-opus-5-5    # Default; use a cheaper model for cost savings
AI_TIMEOUT_MS=10000
AI_MAX_CALLS_PER_NUMBER_PER_HOUR=20
```

The AI responds with structured JSON indicating the detected intent (`flow` and `state`). The bot verifies the output against an allowed schema before trusting it. Invalid AI responses fall back to the default "unrecognized message" reply.

7.6 Maps & Discovery (OpenStreetMap)

Finds nearby blood banks and hospitals for institution outreach:

```
DISCOVERY_PROVIDER=openstreetmap
DISCOVERY_CONTACT_EMAIL=coordinator@yourdomain.com   # Required by OSM usage
```

```
policy
# Optional overrides:
OVERPASS_URL=https://overpass-api.de/api/interpreter
NOMINATIM_URL=https://nominatim.openstreetmap.org
DISCOVERY_TIMEOUT_MS=60000
```

The discovery job: 1. Geocodes the hospital city/address using Nominatim 2. Queries Overpass API for `amenity=blood_bank` and `amenity=hospital` within `OUTREACH_RADIUS_KM` 3. Optionally fetches institution websites for contact enrichment 4. Creates `Institution` records with GeoJSON locations 5. Triggers outreach email to discovered institutions

7.7 Webhooks

WhatsApp Cloud API Webhook: - URL: `https://yourdomain.com/api/v1/webhooks/whatsapp` - Method: `GET` (verification) + `POST` (events) - Verification: responds to Meta's `hub.challenge` with the matching token - Security: validates `X-Hub-Signature-256` HMAC using `WHATSAPP_APP_SECRET` - Events: processes `messages` and `statuses` (delivery receipts)

Setup in Meta Dashboard: 1. Go to App Dashboard → WhatsApp → Configuration → Webhooks 2. Enter Callback URL: `https://yourdomain.com/api/v1/webhooks/whatsapp` 3. Enter Verify Token: same value as `WHATSAPP_WEBHOOK_VERIFY_TOKEN` 4. Subscribe to: `messages`

---

8. Development Guide

8.1 Architecture & Data Flow

Emergency Request Lifecycle

```
Requester submits form / WhatsApp message
         |
         ▼
POST /api/v1/requests
  → Encrypt PII (patient name, contact phone)
  → Create EmergencyRequest (status: "pending")
  → Create AuditLog entry
         |
         ▼
Admin reviews request at /admin/requests
  → PATCH /api/v1/admin/requests/:id/status  (status: "active")
  → If COORDINATION_AUTOSTART=true, triggers matching job
         |
         ▼
Jobs worker: Matching run
  → Query donors with matching blood group within zones
  → Filter: last donation > 90 days, available, consent given
  → Create MatchingRun record
  → Queue donor wave alerts in notification_events
         |
         ▼
WhatsApp worker: Dispatch loop
  → Reads pending notification_events
```

```
  → Sends WhatsApp message (with rate limiting + consent re-check)
  → Records delivery in notification_events
         |
         ▼
Donor receives alert, replies via WhatsApp bot
  → Bot processes inbound message → donor_response flow
  → Records RequestResponse (status: "interested" or "unavailable")
  → Admin sees response at /admin/requests/:id
         |
         ▼
Admin reveals donor contact (if needed)
  → POST /api/v1/requests/:id/donors/:responseId/reveal
  → Decrypts donor phone (logs ContactAccessLog)
         |
         ▼
Request fulfilled / cancelled → status: "fulfilled" / "cancelled"
  → After REQUEST_RETENTION_DAYS, PII is anonymized
```

8.2 API Reference

All API routes are under `/api/v1/`. All responses are JSON.

Authentication header (for user and admin routes): The session cookie `auth-token` is set automatically after login. Admin routes additionally require `role: admin | super_admin`.

Auth

Method	Path	Description
`POST`	`/api/v1/auth/otp/request`	Request an OTP for the given phone number
`POST`	`/api/v1/auth/otp/verify`	Verify OTP and create a session
`POST`	`/api/v1/auth/mfa/verify`	Verify TOTP code (admin MFA)
`POST`	`/api/v1/auth/mfa/enroll`	Begin MFA enrollment (admin)
`POST`	`/api/v1/auth/mfa/enroll/confirm`	Confirm MFA enrollment with a valid TOTP code
`GET`	`/api/v1/auth/me`	Get current user info
`POST`	`/api/v1/auth/logout`	Invalidate session

Donor

Method	Path	Description
GET	/api/v1/donors/me	Get current donor profile
PUT	/api/v1/donors/me	Update donor profile
PUT	/api/v1/donors/me/availability	Toggle availability for requests
PUT	/api/v1/donors/me/consents	Update notification consent
GET	/api/v1/donors/me/notifications	Get notification settings
POST	/api/v1/donors/me/phone/otp	Request phone change OTP
POST	/api/v1/donors/me/phone/verify	Verify new phone number

Requests (public)

Method	Path	Description
POST	/api/v1/requests	Submit a new emergency blood request
GET	/api/v1/requests/:publicRequestId	Get request status (share link view)
POST	/api/v1/requests/:publicRequestId/cancel	Cancel a request (requester only)
POST	/api/v1/requests/:publicRequestId/share-links	Create a tracking share link
GET	/api/v1/requests/:publicRequestId/share-links/:linkId	Get share link
POST	/api/v1/requests/:publicRequestId/contact-links	Create contact link for donor
POST	/api/v1/requests/:publicRequestId/donors/:responseId/reveal	Reveal donor contact details
POST	/api/v1/requests/:publicRequestId/donors/:responseId/contacted	Mark donor as contacted

Admin requests

Method	Path	Description
GET	/api/v1/admin/requests	List all requests
GET	/api/v1/admin/requests/:id	Get full request details
PATCH	/api/v1/admin/requests/:id/status	Change request status
POST	/api/v1/admin/requests/:id/matching	Trigger donor matching run
POST	/api/v1/admin/requests/:id/alerts	Queue donor alert wave
POST	/api/v1/admin/requests/:id/automation	Control automation (start/pause/stop)
GET	/api/v1/admin/requests/:id/coordination	Get coordination timeline
PATCH	/api/v1/admin/requests/:id/institution	Link institution to request
POST	/api/v1/admin/requests/:id/outreach/:outreachId	Retry institution outreach

Admin institutions

Method	Path	Description
GET	/api/v1/admin/institutions	List institutions
POST	/api/v1/admin/institutions	Create institution manually
GET/PATCH	/api/v1/admin/institutions/:id	Get / update institution
PATCH	/api/v1/admin/institutions/:id/status	Change institution status
POST	/api/v1/admin/institutions/:id/review	Review / approve institution
GET/POST	/api/v1/admin/institutions/:id/contacts	List / add contacts
GET/PATCH/DELETE	/api/v1/admin/institutions/:id/contacts/:contactId	Manage contact
PATCH	/api/v1/admin/institutions/:id/channels/:channelId/status	Enable/disable channel

Admin operations

Method	Path	Description
GET	/api/v1/admin/operations	Get operation stats dashboard
GET/PATCH	/api/v1/admin/operations/settings	Get / update system settings (alerts paused, bot enabled)
GET	/api/v1/admin/operations/failed-messages	List failed outbound messages
POST	/api/v1/admin/messages/:messageId/retry	Retry a failed message
GET	/api/v1/admin/audit	Audit log
GET	/api/v1/admin/conversations	Active WhatsApp conversations

Admin WhatsApp (Web pilot)

Method	Path	Description
`POST`	`/api/v1/admin/whatsapp/connect`	Trigger QR code generation
`GET`	`/api/v1/admin/whatsapp/qr`	Stream QR code
`GET`	`/api/v1/admin/whatsapp/status`	Connection status
`POST`	`/api/v1/admin/whatsapp/logout`	Disconnect WhatsApp session
`POST`	`/api/v1/admin/whatsapp/send-message`	Send a manual test message

Health

Method	Path	Description
`GET`	`/api/health`	Liveness + DB readiness. Returns `{"status":"ok","db":"up"}` or 503

8.3 Authentication

Session flow: 1. OTP request → OTP hashed and stored → delivered via WhatsApp/SMS/console 2. OTP verify → JWT generated → stored in `sessions` → HTTP-only cookie set 3. Admin MFA → TOTP verified → MFA session flag set 4. Every API request → JWT verified → session checked in DB → user loaded

Cookie: `auth-token` (HTTP-only, Secure in production, SameSite=Lax)

Permissions: | Role | Access | |—|—| | `donor` | Own profile, public requests | | `coordinator` | Same as donor + limited admin views | | `institution_user` | Institution-specific views | | `super_admin` / `admin` | Full admin dashboard |

Admin routes use `requireAdmin()` middleware. User routes use `requireUser()`.

8.4 Background Workers

WhatsApp Worker (`npm run whatsapp:worker`)

- Owns the WhatsApp Web browser session via MongoDB lease
- Polls MongoDB `notification_events` for pending messages
- Processes inbound WhatsApp messages through the bot conversation engine
- Exposes a loopback control server on port 3100 for admin commands (connect, disconnect, QR, status)
- Performs heartbeat lease renewal every 30 seconds
- On `SIGTERM`: gracefully drains, marks connection as disconnected, releases lease

Key state machine files: - `src/server/conversations/engine.ts` — routes messages to the right flow - `src/server/conversations/onboarding-machine.ts` — donor registration

flow - `src/server/conversations/request-machine.ts` — emergency request flow - `src/server/conversations/bot.ts` — message rendering and AI integration

Jobs Worker (`npm run jobs:worker`)

- **Stateless** — multiple instances can run safely (jobs are claimed atomically via `findOneAndUpdate`)
- Polls the `jobs` collection every 3 seconds
- **Job types handled:**
  - `institution_discovery` — OpenStreetMap search for nearby blood banks
  - `institution_outreach_email` — Sends outreach emails to institutions
  - `donor_wave` — Alerts a batch of donors in a zone
  - `request_coordination` — Schedules next wave or marks coordination complete
- Runs data retention anonymization once per hour
- With `MESSAGING_PROVIDER=whatsapp_cloud`: also drains the notification outbox (WhatsApp worker not needed)

8.5 Security Headers

Configured in `next.config.ts` for all routes:

Header	Value
`Content-Security-Policy`	`default-src 'self'; script-src 'self' 'unsafe-inline'; ...`
`X-Content-Type-Options`	`nosniff`
`X-Frame-Options`	`DENY`
`Referrer-Policy`	`strict-origin-when-cross-origin`
`Permissions-Policy`	`camera=(), microphone=(), geolocation=(self), payment=()`
`Strict-Transport-Security`	`max-age=63072000; includeSubDomains; preload` (production only)
`X-Powered-By`	Removed (`poweredByHeader: false`)

Tracking and respond URLs (`/track/`, `/respond/`) additionally set: - `Referrer-Policy: no-referrer` - `X-Robots-Tag: noindex, nofollow`

---

9. Testing

Run tests

```
# All tests (unit + integration)
# Requires MongoDB running at localhost:27017
npm run test:integration

# Unit tests only (no MongoDB needed)
npm test

# Type check
```

```
npm run typecheck

# ESLint
npm run lint

# Production build
npm run build
```

Test suite structure

File	Type	What it tests
`tests/crypto.test.ts`	Unit	PII encryption, keyed hashing, TOTP
`tests/phone.test.ts`	Unit	Indian mobile number normalization
`tests/session-dir.test.ts`	Unit	WhatsApp session directory resolution
`tests/onboarding-machine.test.ts`	Unit	Bot conversation state machine
`tests/sprint-units.test.ts`	Unit	Core business logic units
`tests/sprint2-units.test.ts`	Unit	Sprint 2 unit coverage
`tests/phase5-units.test.ts`	Unit	Phase 5 (institution, outreach) units
`tests/phase6-units.test.ts`	Unit	Phase 6 (jobs, automation) units
`tests/otp-flow.integration.test.ts`	Integration	Full OTP sign-in flow
`tests/donor.integration.test.ts`	Integration	Donor registration, profile, consent
`tests/requests.integration.test.ts`	Integration	Emergency request lifecycle
`tests/messaging.integration.test.ts`	Integration	Outbox, dispatch, rate limiting
`tests/cloud-api.integration.test.ts`	Integration	WhatsApp Cloud API webhook handling
`tests/sprint2-security.integration.test.ts`	Integration	Auth security, session management
`tests/phase5-workflows.integration.test.ts`	Integration	Institution + outreach flows
`tests/phase6-workflows.integration.test.ts`	Integration	Job queue, donor waves, coordination
`tests/backup-restore.integration.test.ts`	Integration	Backup and restore drill

Test environment

Tests in `tests/setup.ts` generate their own random secrets (no real credentials needed). Integration tests use `MONGODB_URI_TEST=mongodb://127.0.0.1:27017/marutix_test` to avoid touching the development database.

No real WhatsApp messages, emails, or AI calls are made during tests.

Dependency security audit

```
# Check package signatures (verify supply chain)
npm audit signatures

# Run the custom dependency gate (checks high/critical advisories)
node scripts/audit-gate.mjs
```

---

10. CI/CD & Deployment

Three-stage GitHub Actions pipeline

Stage 1 — Code Quality (`code-quality.yml`)

Runs on every push, pull request, and weekly schedule:

Job	What it checks
ESLint	TypeScript/React lint rules
TypeScript	Full type check (`next typegen && tsc --noEmit`)
Tests	Full integration test suite with MongoDB 9.0.4
Dependency audit	npm signature verification + custom allowlist gate
Dependency review	PRs: flags new high/critical advisory additions
CodeQL	SAST scan (JS/TS + Actions workflows)

Stage 2 — Build & Push (`build-push.yml`)

Runs automatically after Stage 1 succeeds on `main` or a release tag:

1. **Resolve commit** — determines SHA, kind (branch/tag), whether to push
2. **Next.js build** — full production build
3. **Docker images** — builds `marutix-web` and `marutix-worker` in parallel
4. **Smoke tests** — verifies each image (non-root, no `.env` leaks, health endpoint, security headers)
5. **Push to GHCR** — pushes with SHA tag + branch/version tags (only on real pushes)
6. **Release manifest** — uploads `deploy-manifest.json` artifact with digests

Images are pushed to: - `ghcr.io/piyush-sharma788/marutix-web:<sha>` - `ghcr.io/piyush-sharma788/marutix-worker:<sha>`

Stage 3 — Deploy (`deploy.yml`)

Runs after Stage 2 produces a manifest:

1. **Plan** — downloads manifest, selects environment (staging/production)
2. **Deploy** — SSH to server, copy Compose file + deploy script, verify image digests, `docker compose up -d`
3. **Health check** — polls `$APP_URL/api/health` until `{"status":"ok","db":"up"}`
4. **Auto-rollback** — on failure, re-runs compose with the previous tag

Required GitHub Secrets & Variables

See `CI_CD_SETUP.md` for the full setup guide. Summary:

Environment secrets (set in both `staging` and `production`): - `SSH_PRIVATE_KEY` — Ed25519 private key for server access - `SSH_KNOWN_HOSTS` — Server fingerprints (from `ssh-keyscan`) - `GHCR_READ_TOKEN` — GitHub PAT with `read:packages` scope

Repository variables: - `WHATSAPP_WEB_PUBLIC_NUMBER` — Baked into Docker image at build time - `STAGING_DEPLOY_ENABLED=true` — Enable automatic staging deploys - `PRODUCTION_DEPLOY_ENABLED=true` — Enable production deploys on tags - `GHCR_READ_USER` — GitHub username

Environment variables (per staging/production): - `DEPLOY_HOST`, `DEPLOY_USER`, `DEPLOY_PATH`, `APP_URL`

Production server setup

See `CI_CD_SETUP.md` for step-by-step instructions. Quick summary:

```
# 1. Install Docker on the server
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $DEPLOY_USER

# 2. Create deployment directory
sudo mkdir -p /opt/marutix
sudo chown deploy:deploy /opt/marutix

# 3. Create .env.production (see §3.1 for all variables)
nano /opt/marutix/.env.production
chmod 600 /opt/marutix/.env.production

# 4. Set up reverse proxy (Caddy — easiest for HTTPS)
# Install Caddy: https://caddyserver.com/docs/install
cat > /etc/caddy/Caddyfile << 'EOF'
marutix.example.com {
    reverse_proxy 127.0.0.1:3000
}
EOF
systemctl restart caddy

# 5. First manual deploy (from your machine)
# Trigger via GitHub Actions → Build & Push → Run workflow → main
# Then → Deploy → Run workflow → production
```

Release process

```
# Tag a release
git tag v1.0.0
git push origin v1.0.0
# → Triggers Code Quality → Build & Push → Deploy (production)
```

Manual rollback

```
# On the server
cd /opt/marutix
PREV=$(cat .previous-tag)
MARUTIX_IMAGE_PREFIX=ghcr.io/piyush-sharma788 \
MARUTIX_IMAGE_TAG="$PREV" \
docker compose -f docker-compose.yml up -d
```

---

11. Maintenance & Operations

11.1 Logging

All processes use structured JSON logging. Each log line is a JSON object with:

```
{
  "level": "info",
  "time": "2026-11-05T06:00:00.000Z",
  "event": "request.created",
  "requestId": "REQ-123456",
  "bloodGroup": "O+",
  "city": "Mumbai"
}
```

Sensitive data is never logged — phone numbers, patient names, and other PII are redacted before logging.

```
# View web app logs (Docker)
docker compose -f deploy/docker-compose.prod.yml logs -f web

# View worker logs
docker compose -f deploy/docker-compose.prod.yml logs -f worker

# View jobs worker logs
docker compose -f deploy/docker-compose.prod.yml logs -f jobs

# Filter for errors only
docker compose -f deploy/docker-compose.prod.yml logs web | grep
'"level":"error"'

# Tail logs with jq formatting (if jq installed)
docker compose -f deploy/docker-compose.prod.yml logs -f web | while read
line; do
  echo "$line" | jq . 2>/dev/null || echo "$line"
done
```

Configure an error webhook for automatic error alerting:

```
ERROR_ALERT_WEBHOOK_URL=https://hooks.slack.com/services/xxx/yyy/zzz
```

11.2 Health Checks

Application health:

```
curl https://marutix.example.com/api/health
# Expected: {"status":"ok","db":"up"}
# Failure: {"status":"unavailable","db":"down"} with HTTP 503
```

Container health:

```
docker inspect --format='{{.State.Health.Status}}' marutix-web-1
# Expected: healthy
```

WhatsApp worker health (from admin dashboard): - `/admin/whatsapp` shows connection state, last seen, account number - API: `GET /api/v1/admin/whatsapp/status`

Admin operations dashboard: - `/admin/operations` shows: bot enabled/disabled, alerts paused/active, failed messages count

11.3 Backups

Automated backup (cron job):

```
# Add to root or deploy user crontab
crontab -e

# Run backup daily at 3 AM UTC
0 3 * * * \
  MONGODB_URI="$(grep MONGODB_URI /opt/marutix/.env.production | cut -d= -
f2-)" \
  BACKUP_DIR=/var/backups/marutix \
  BACKUP_ENCRYPTION_PASSPHRASE_FILE=/etc/marutix-backup.key \
  /opt/marutix/scripts/backup.sh >> /var/log/marutix-backup.log 2>&1
```

Check backup health:

```
# Latest backup
ls -lh /var/backups/marutix/marutix-*.archive.gz | tail -1

# Verify checksum
cd /var/backups/marutix
sha256sum -c marutix-20261105T030000Z.sha256

# Check collection counts in manifest
cat marutix-20261105T030000Z.manifest.json | python3 -m json.tool
```

Offsite backup: Copy backups to S3, Google Cloud Storage, or any offsite location. The encrypted archives are safe to store anywhere — they are useless without `BACKUP_ENCRYPTION_PASSPHRASE_FILE`.

```
# Example: sync to S3
aws s3 sync /var/backups/marutix s3://your-bucket/marutix-backups/ \
  --exclude "*" --include "*.enc" --include "*.sha256" --include
"*.manifest.json"
```

11.4 Admin Operations

Grant/revoke admin role:

```
# Production (run on the server inside the worker container)
docker compose -f /opt/marutix/docker-compose.yml exec worker \
  node_modules/.bin/tsx --conditions=react-server scripts/admin-role.ts grant
+919876543210

# Revoke
docker compose exec worker \
  node_modules/.bin/tsx --conditions=react-server scripts/admin-role.ts revoke
+919876543210

# List all admins
docker compose exec worker \
  node_modules/.bin/tsx --conditions=react-server scripts/admin-role.ts list

# Reset MFA (if admin is locked out)
docker compose exec worker \
  node_modules/.bin/tsx --conditions=react-server scripts/admin-role.ts mfa-
reset +919876543210
```

System settings (from admin dashboard): - **Pause alerts** — Stop all WhatsApp donor alerts temporarily (e.g., for maintenance) - **Disable bot** — Stop processing inbound WhatsApp messages

Retry failed messages: - Admin → Operations → Failed Messages → Retry - Or API: `POST /api/v1/admin/messages/:messageId/retry`

11.5 Common Errors

Error message	Cause	Fix
`MarutiX web refused to start. Fix these settings: - core: MONGODB_URI is required`	Missing required env variable	Add the named variable to `.env.local` or `.env.production`
`PII_ENCRYPTION_KEY must be 32 bytes encoded as base64`	Wrong format for encryption key	Generate: `openssl rand -base64 32` — the result must be ~44 chars
`AUTH_JWT_SECRET has too few distinct characters to be random`	Using a placeholder like "changeme"	Generate: `openssl rand -hex 64`
`AUTH_JWT_SECRET and OTP_HASH_SECRET must be different secrets`	Reused the same value	Generate separate values for each
`The development OTP provider can never be used in production`	`OTP_PROVIDER=development` with `NODE_ENV=production`	Configure WhatsApp OTP or Twilio SMS
`MongoDB cannot start: Linux kernel versions 6.19 and newer`	Old MongoDB version in Docker	Update to `mongo:9.0.4` (CI is already fixed)
`Another WhatsApp worker is running. Exiting.`	Two worker processes started	Stop the duplicate; check for zombie processes
`whatsapp.worker.lease_lost`	Worker was restarted while another instance took the lease	Expected; the new instance will try to acquire the lease
`Error: WHATSAPP_GRAPH_API_VERSION must look like v26.0`	Bad env value format	Must be exactly `v26.0`, not `26` or `v26.0.0`
`backup: the database has no collections`	Wrong `MONGODB_URI` or database name	Verify the URI points to the correct database
`OTP_PROVIDER=whatsapp requires WHATSAPP_ACCESS_TOKEN, ...`	WhatsApp OTP configured but credentials missing	Either add all 3 WhatsApp vars or set `OTP_PROVIDER=development`
Container health status: `unhealthy`	App is not responding to health checks	Check logs for startup errors; verify MongoDB is reachable
`rate limit exceeded` from Meta	Too many WhatsApp messages sent	Reduce `WHATSAPP_CLOUD_MAX_MESSAGES_PER_HOUR` or add delay
CodeQL job fails with `Resource not accessible`	Private repo without GitHub Advanced Security	Expected on GitHub Free; either make repo public or disable CodeQL job

This document was generated from the actual MarutiX codebase on 2026-11-05. It reflects the state of the code at commit `c93d917`.

For CI/CD setup, see `CI_CD_SETUP.md`. For the CI/CD verification report, see `CI_CD_VERIFICATION_REPORT.md`.